

Document number: P4149R1
Date: 2026-03-27
Audience: EWG, CWG
Reply-to: Andrzej Krzemiński <akrzemi1 at gmail dot com>
Brian Bi <bbi5291 at gmail dot com>

Define "immediate context"

This paper aims to provide the definition for the term "immediate context". This addresses [\[US54-100\]](#).

1. EWG guidance

1.1. Round 1

The following three polls have been taken on 2026-03-24 during the Croydon meeting.

Poll 1: As an answer to US 54-100 for CWG, default arguments ARE part of the immediate context

SF	F	N	A	SA
0	2	12	10	6

Verdict: not consensus.

Poll 2: As an answer to US 54-100 for CWG, default arguments ARE NOT part of the immediate context

SF	F	N	A	SA
5	12	11	2	1

Verdict: consensus.

Poll 3: Opinion: default member initializers of aggregates should be part of the immediate context. Consensus may be called either for/against.

SF	F	N	A	SA
1	1	12	10	3

Verdict: Initially incorrectly declared "no consensus", then corrected to "consensus against".

1.2. Round 2

CWG would like to unify the treatment of noexcept-specifiers, function-contract-specifiers, default arguments, and annotations in order to improve the consistency of the language and the specification. The proposed unified treatment is that the immediate context excludes separately instantiated constructs and includes everything else (other than lambda bodies). To that end, CWG is requesting EWG's consent for the following changes/clarifications. We believe that items 1 and 2 below apply only to the special case of constructs that appear in a non-generic lambda in a SFINAE context (like the return type of a function template).

1.2.1. Item 1

Default arguments are always subject to separate instantiation, even when they appertain to non-generic lambdas or member functions of local classes. However, in non-generic lambdas and member functions of local classes, that separate

instantiation is not deferred; consequently, default arguments are excluded from the immediate context even in these cases. (For local classes, this is not a change in behavior. For non-generic lambdas, the status quo is unclear). For example:

```
struct X {};  
  
// default argument not used:  
template <class T>  
auto foo() -> decltype([](T x = T(1)){}(T()));  
  
template <class T>  
concept canFoo = requires { foo<T>(); };  
  
constexpr bool b = canFoo<X>; // ???
```

CWG suggestion: There is no separate instantiation for the default argument in this case. A hard error therefore occurs during overload resolution, even though the default argument isn't used.

Implementation status: GCC currently accepts; Clang and MSVC reject.

1.2.2. Item 2

noexcept-specifiers on function declarations will be changed so that they are treated the same way as suggested above for default arguments, i.e., to receive separate instantiation always (and thus never be in the immediate context) but have that separate instantiation be performed immediately in the non-generic lambda and local class cases. (The status quo is that *noexcept-specifiers* of non-generic lambdas and member functions of local classes are not subject to separate instantiation. In the lambda case, Clang and GCC treat them as being in the immediate context.) For example:

```
template <class T>  
auto foo() -> decltype([]() noexcept(noexcept(T() + 1)){ /* ... */ });  
  
template <class T>  
concept canFoo = requires { foo<T>(); };  
  
struct X {};  
  
constexpr bool b = canFoo<X>; // ???
```

Status quo: `b` is initialized to `false`.

Implementation status: Clang and GCC agree, while MSVC gives a hard error.

CWG suggestion: the result should be a hard error because the *noexcept-specifier* is immediately but separately instantiated.

If this suggestion is rejected, then *noexcept-specifiers* will be self-consistent (not separately instantiated in the above context + yes in the immediate context) but different from function contract specifiers.

1.2.3. Item 3

Annotations of templated entities should be instantiated only when needed, similarly to *noexcept-specifiers* and function contract specifiers, and are therefore never in the immediate context. (CWG has not yet formulated a position as to when, exactly, annotations are considered needed.) Unlike in items 1 and 2, this clarification would affect templated functions in general, not just the lambda edge cases.

1.2.4. The polls

On 2026-03-26 during the Croydon meeting, EWG took the following polls.

Poll 4: Agree with P4149R0 and forward the core position for Item #1 and #2 to CWG as a (partial) fix for US54-100

SF	F	N	A	SA
3	15	3	0	0

Verdict: consensus.

Poll 5: For P4149R0 Item #3: Annotations should be part of the immediate context

SF	F	N	A	SA
0	4	7	4	2

Verdict: not consensus.

Poll 6: Agree with P4149R0 and forward the core position for Item #3 to CWG as a (partial) fix for US54-100

SF	F	N	A	SA
2	10	6	0	0

Verdict: consensus.

1.3. Round 3

CWG asks for consent from EWG for the following changes/clarifications:

1. Annotations are separately instantiated constructs when they appertain to templated functions or parameters thereof.
2. Annotations are instantiated "as needed". They will be considered "needed" using the same rules as noexcept-specifiers:
 - immediately, in the case of lambdas and members of local classes,
 - when the function is selected by overload resolution,
 - when the function is odr-used,
 - when the function is defined, or
 - when the function is represented by a reflection. (This bullet is currently missing for noexcept-specifiers; CWG considers this a defect and will fix it in the wording.)

The rationale for the above rule is primarily consistency: a "just in time" instantiation rule for annotations would be unlike anything else in the language. The question arose of whether to be consistent with noexcept-specifiers or with contracts. It makes more sense to be consistent with noexcept-specifiers since annotations are a compile-time facility that are not inherently tied to evaluated calls. For simplicity of specification and teachability, parameter annotations should be instantiated at the same time as annotations of functions themselves.

Poll 7: Instruct core to resolve US 54-100 without specifying when annotations are instantiated

SF	F	N	A	SA
3	17	2	0	0

Verdict: consensus.

3. Wording

The proposed wording is relative to [\[N5032\]](#).

Edit [except.spec]/12:

An exception specification is considered to be *needed* when:

- in an expression, the function is selected by overload resolution ([over.match], [over.over]);
- the function is odr-used ([basic.def.odr]);
- the exception specification is compared [...];
- the function is defined; ~~or~~
- the function is represented by a reflection; or
- the exception specification is needed for a defaulted function that calls the function. [Note: ...]

The exception specification of a defaulted function is evaluated as described only when needed; ~~similarly, the noexcept-specifier of a specialization of a templated function is instantiated only when needed.~~

Edit [temp.decls.general]/3:

A separately instantiated construct of a templated function F is a

- default argument,
- noexcept-specifier, or
- function-contract-specifier

of F.

For purposes of name lookup and instantiation, ~~default arguments~~ separately instantiated constructs, type-constraints, and requires-clauses ([temp.pre]), ~~and noexcept-specifiers, of function templates and of member functions of class templates~~ are considered definitions; each ~~default argument~~ separately instantiated construct, type-constraint, or requires-clause ~~or noexcept-specifier~~ is a separate definition which is unrelated to the templated function definition or to any other ~~default arguments~~ separately instantiated constructs, type-constraints, or requires-clauses, ~~or noexcept-specifiers~~. For the purpose of instantiation, the substatements of a constexpr if statement are considered definitions. For the purpose of name lookup and instantiation, the compound-statement of an expansion-statement is considered a template definition.

Strike note 3 from [temp.inst]/2:

~~[Note 3: Within a template declaration, a local class or enumeration and the members of a local class are never considered to be entities that can be separately instantiated (this includes their default arguments, noexcept-specifiers, and non-static data member initializers, if any, but not their type-constraints or requires-clauses). As a result, the dependent names are looked up, the semantic constraints are checked, and any templates used are instantiated as part of the instantiation of the entity within which the local class or enumeration is declared. —end note]~~

Edit [temp.inst]/3:

~~[...] The implicit instantiation of a class template specialization does not cause the implicit instantiation of default arguments or noexcept-specifiers of the class member functions: [...]~~

Edit [temp.inst]/5:

Unless a function template specialization is a declared specialization, the function template specialization is implicitly instantiated when the specialization is referenced in a context that requires a function definition to exist or if the existence of the definition affects the semantics of the program. A function whose declaration was instantiated from a friend function definition is implicitly instantiated when it is referenced in a context

that requires a function definition to exist or if the existence of the definition affects the semantics of the program. ~~Unless a call is to a function template explicit specialization or to a member function of an explicitly specialized class template, a default argument for a function template or a member function of a class template is implicitly instantiated when the function is called in a context that requires the value of the default argument.~~

Add a paragraph before [temp.inst]/12:

The separately instantiated constructs ([temp.decls.general]) of a templated function F that is either

- a member of a local class or
- the function call operator of the closure type of a non-generic *lambda-expression*

are instantiated when the declaration of F is instantiated.

[Note: For the purposes of instantiation, these constructs are still considered separately from the function to which they belong. — end note]

Edit [temp.inst]/12:

Other than as specified above, when ~~if~~ a templated function f is called in a way that requires a default argument to be used, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the default argument is done as if the default argument had been an initializer used in a function template specialization with the same scope, the same template parameters and the same access as that of the function template f used at that point, except that the scope in which a closure type is declared ([expr.prim.lambda.closure]) — and therefore its associated namespaces — remain as determined from the context of the definition for the default argument. This analysis is called default argument instantiation. The instantiated default argument is then used as the argument of f.

Edit [temp.inst]/13:

[Note: Each default argument is instantiated independently. — end note]

[Example 8: ...]

Edit [temp.inst]/14:

Other than as specified above, the ~~The noexcept-specifier and function-contract-specifiers~~ separately instantiated constructs of a ~~function template~~ specialization of a templated function are ~~not instantiated along with the function declaration; they are~~ instantiated only when needed ([except.spec], [dcl.contract.func]). ~~if~~ When such a specifier construct is ~~needed but has not yet been~~ instantiated, the dependent names are looked up, the semantics constraints are checked, and the instantiation of any template used in the specifier construct is done as if it were being done as part of instantiating the declaration of the specialization at that point.

Edit [temp.deduct.general]/8 as follows.

If a substitution results in an invalid type or expression, type deduction fails. An invalid type or expression is one that would be ill-formed, with a diagnostic required, if written in the same context using the substituted arguments.

[Note: ...]

Invalid types and expressions can result in a deduction failure only in the immediate context of the deduction substitution loci. The immediate context of a deduction substitution locus is that deduction substitution locus, excluding bodies of lambda-expressions and separately instantiated constructs.

[Example:

```
template<typename T>
T* fun(T&& v); // #1: the deduction substitution locus is the function type
               // "function of (rvalue reference to T) returning pointer to T"

void fun(...); // #2

void test().
{
    int i;
    fun(i); // selects #2 (forming the type "pointer to reference to int" fails in #1)
}
```

— end example]

[Note: Separately instantiated constructs are excluded from the immediate contexts even if they are instantiated at the same time. — end note]

[Note 6: The substitution into types and expressions can result in effects such as the instantiation of **class** template specializations **and/or function template specializations**, the generation of implicitly-defined functions, etc. Such effects are not in the "immediate context" and can result in the program being ill-formed. — end note]

Strike the normative text in [temp.deduct.general]/9, update the note to more concisely express the rationale, and leave the example (demonstrating the interaction of lambdas with SFINAE):

When substituting into a lambda-expression, substitution into its body is not in the immediate context.

[Note 7: **The intent is to avoid requiring implementations to deal with substitution failure involving arbitrary statements** No statement is ever in the immediate context of a deduction substitution locus.

[Example 7: ... —end example]

—end note]

4. References

- [CWG2296] — Jens Maurer, "C++ Standard Core Language Active Issues, Revision 111", issue #2296, (https://www.open-std.org/jtc1/sc22/wg21/docs/cwg_active.html#2296).
- [N5032] — Thomas Köppe, "Working Draft, Standard for Programming Language C++" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2025/n5032.pdf>).
- [P0348R0] — Andrzej Krzemiński, "Validity testing issues" (<http://www.open-std.org/jtc1/sc22/wg21/docs/papers/2016/p0348r0.html>).
- [P2285R1] — Andrzej Krzemiński, Tomasz Kamiński, "Are default function arguments in the immediate context?" (<https://www.open-std.org/jtc1/sc22/wg21/docs/papers/2026/p2285r1.html>).
- [63391] — llvm/llvm-project GitHub issue, "clang++: Surprising SFINAE behavior for default function arguments" (<https://github.com/llvm/llvm-project/issues/63391>).